

유동연 Frontend Developer

2000.05.02 · 010-2996-1353 · [Blog](#) · [GitHub](#)

좋은 개발은 단순히 코드를 잘 작성하는 것을 넘어, 시간과 비용, 팀의 상황, 유지보수 가능성, 사용자 경험 사이에서 적절한 균형을 찾는 일이라고 생각합니다

코드는 문제를 해결하는 수단이며, 진짜 어려운 일은 문제를 정확히 정의하는 것입니다



PRINCIPLES 일하는 기준

오컴의 면도날	불필요한 복잡성을 걷어내고, 문제를 가장 단순한 형태로 푸는 것을 지향
밀러의 법칙	사람이 한 번에 감당할 수 있는 정보량을 고려해, 읽는 사람 관점에서 코드와 UI를 설계

SKILLS 기술

Language

JavaScript

TypeScript

Library / Framework

React.js

Next.js

상태 · 데이터

TanStack Query

Zustand

Zod

테스트 · 도구

Vitest

MSW

Vite

UI

Tailwind CSS

shadcn/ui

SafeKiosk

2026.05 – 2026.07

무인 키오스크 매장의 매출·고객·시간대 통계를 차트로 보여주는 관리자 대시보드 (사내 POC)

어떤 지표를 어떤 차트로 볼지 계속 바뀌는 상황에서, 차트를 바꿔도 BE 변경이 필요 없도록 통계 응답 계약과 차트 공통 레이어를 설계

React 19

TypeScript

Vite

ECharts

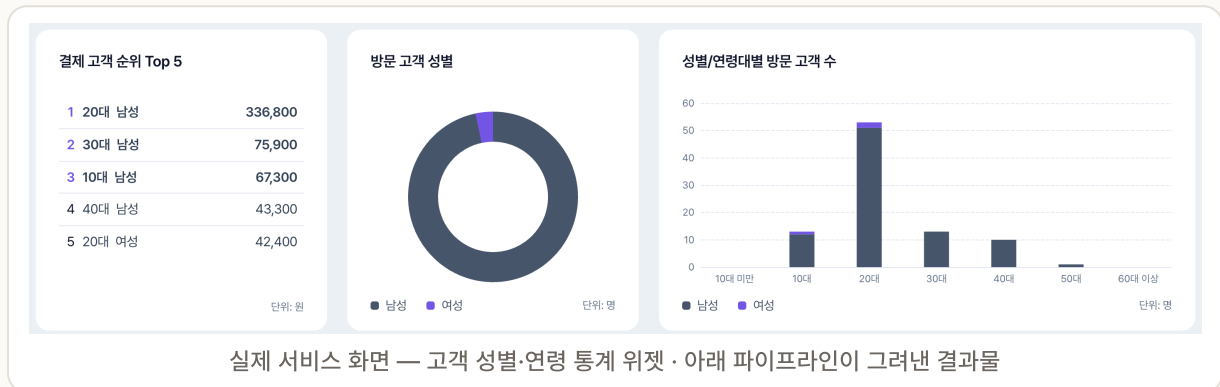
TanStack Query

Zod

Tailwind CSS

Vitest · MSW

ARCHITECTURE 전체 구조 · 데이터 흐름



PROBLEM 문제 원인

1 차트 하나 추가할 때마다 BE와 응답 형상 협의 필요

- 차트별로 전용 응답을 맞추는 방식이라 새 차트마다 "어떤 JSON을 줄지"를 협의해야 함
- 지표·차트 구성이 수시로 바뀌는 POC에서 FE의 차트 실험 속도가 BE 릴리스 일정에 묶임

2 차트 코드가 응답 필드명에 직접 의존

- 축·시리즈로 쓸 필드명이 차트 컴포넌트 안에 하드코딩돼, BE가 필드 하나만 바뀌어도 그 필드를 쓰는 차트를 찾아 고쳐야 함
- API가 11종으로 늘면서 고칠 곳도 같이 늘어남

3 위젯마다 같은 데이터 가공 코드 반복

- 성별·연령·카테고리로 묶고 값을 합산하는 로직은 어떤 차트든 같은데, 공통 지점이 없어 위젯마다 다시 구현하고 수정도 각각 해야 함

SOLUTION 해결 과정



1 모든 통계 API를 단일 응답 계약으로 통일 — tidy(long) format

- 11종 전부 { schema, data } 한 형태로 고정
- data는 한 행이 관측치 1건인 long-format 표, schema는 각 컬럼이 축(dimension)인지 값(measure)인지 알려주는 메타데이터
- 형상이 고정되니 새 지표가 생겨도 BE와 형상을 다시 협의할 필요가 없음

2 차트가 필드명을 모르게 — schema 기반 조회

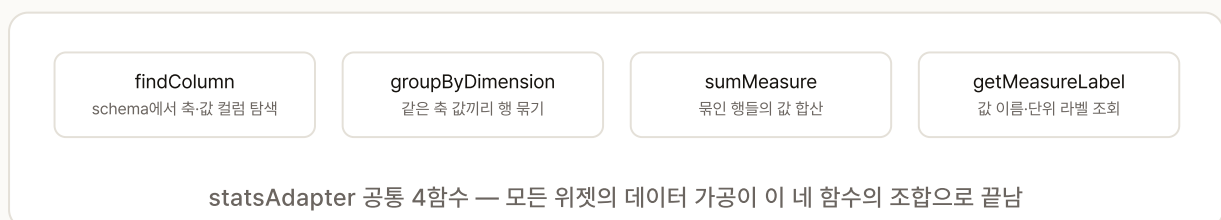
- 축 라벨·단위·컬럼 위치를 코드에 박지 않고 응답 안의 schema에서 읽음
- BE가 컬럼을 바꿔도 차트 코드는 그대로
- 컬럼명 타입을 응답 스키마에서 파생해 오타는 컴파일 단계에서 차단

3 데이터 가공을 공통 어댑터로 수렴

- 컬럼 찾기·그룹핑·합산·라벨 조회 4함수를 statsAdapter 한곳에 모으고, 위젯은 이를 조합한 순수 함수 transform*.ts에서 { categories, series } 뷰모델만 생성
- 차트 컴포넌트는 뷰모델을 받아 그리기만 담당

4 계약이 깨지면 조용히 넘어가지 않게 가드

- 계약을 벗어난 응답은 개발 환경에선 throw로 즉시 실패시키고, 프로덕션에선 warn 후 원본을 통과시킴
- BE 변경을 개발 단계에서 발견하되 사용자 화면은 죽지 않게 함



RESULT 결과

- 1 차트 위젯 **11개**를 BE 응답 형상 재협의 없이 추가 — 새 지표를 그리는 데 필요한 FE 작업이 변환 함수 하나 추가로 수렴, 신규 API 연동도 요청·조회 혹 두 파일이면 완료
- 2 데이터 가공 로직이 공통 어댑터 **4함수**로 수렴 — 변환이 렌더와 분리된 순수 함수라 차트를 띄우지 않고 단위 테스트로 검증
- 3 BE 응답 형상 변경을 개발 단계에서 감지 — 실제 위치 API 형상 확장 때 dev 가드로 잡아 대응하고 회귀 테스트로 고정

Safe Vision

2026.03 – 2026.05

매장 카메라로 결품(OOS)·매대 변화를 감지하는 AI 비전 서비스의 관리자 웹앱

WeWeb 노코드 웹앱을 React로 재구축하고 웹↔네이티브 통신 모듈을 양쪽에서 설계·구현

롯데마트 현장 PoC 투입 · iOS·Android 앱 스토어 출시

React 19

TypeScript

Vite

TanStack Query

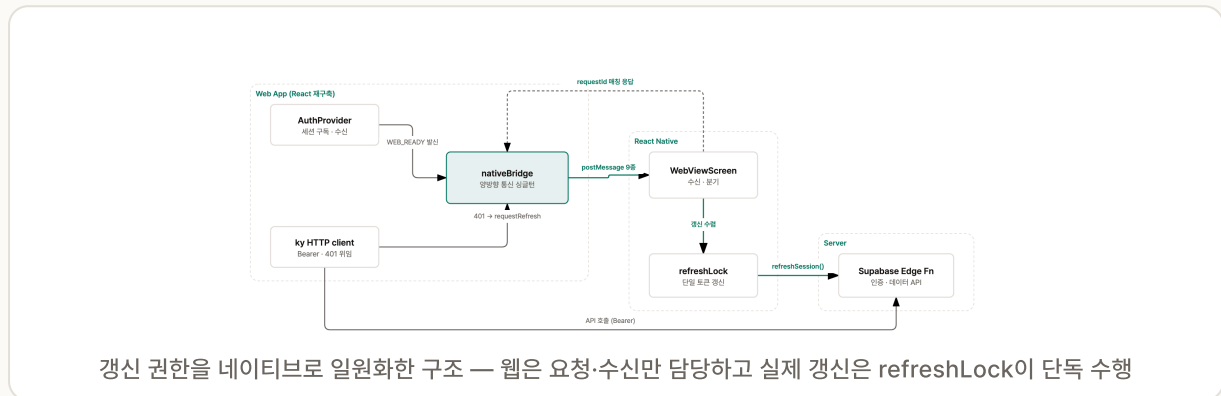
Zustand

ky

Supabase

react-native-webview

ARCHITECTURE 전체 구조 · 데이터 흐름



PROBLEM 문제 원인

1 노코드 런타임의 성능·유지보수 한계

- WeWeb 런타임·에디터 번들이 통째로 전송돼 동일 화면 기준 JS **3,457KB** · LCP **19.27s**
- 생명주기·실행 순서를 추적할 수 없어 WebView 연동 디버깅이 어려움

2 웹·네이티브 초기화 순서 미보장

- 네이티브의 세션 주입과 웹의 리스너 등록 순서가 실행마다 달라짐
- 웹뷰 로드 완료 이벤트로는 리스너 등록 시점을 알 수 없는 것이 원인, 네이티브가 먼저 쏘면 웹은 미인증 상태로 진입

3 토큰 갱신 요청·응답의 대응 부재

- 여러 API가 동시에 401을 받으면 각각 갱신을 요청하는데, 응답에 어느 요청의 것인지 식별자가 없어 매칭 불가
- 응답이 유실되면 무한 대기

SOLUTION 해결 과정

1 React 재구축 + nativeBridge 싱글턴 설계

- 웹앱 전체를 React·Vite로 재작성하고 웹↔네이티브 통신을 싱글턴 모듈 하나로 수렴
- 이벤트 9종·payload 타입을 레지스트리 한곳에 모아 잘못된 호출을 컴파일 단계에서 차단, 브라우저에선 no-op으로 동작

2 WEB_READY 핸드셰이크

- 웹이 리스너 등록을 마친 직후 준비 완료를 선언해야 네이티브가 세션을 주입
- 순서를 타이밍이 아닌 수신 측 선언으로 보장

3 requestId 매칭 + 갱신 권한 네이티브 일원화

- 갱신 요청마다 requestId를 부여해 자기 요청의 응답만 수락, push/response는 kind 필드로 구분
- 실제 refreshSession()은 네이티브 refreshLock(단일 Promise + 쿨다운 캐시)만 수행, 동시 호출은 1회 수렴 · 10초 타임아웃으로 무한 대기 차단

RESULT 결과

지표	WeWeb	React 재구축	개선
LCP	19.27s	3.29s	-83%
콘텐츠 표시 완료	17.28s	3.29s	-81%
JS 전송량	3,457KB	1,240KB	-64%
JS 힙 사용량	64.9MB	18.4MB	-72%
Long Task 수	8개	2개	-75%

측정: 로그인 후 동일 대시보드 · Playwright Chromium · 4x CPU 스로틀 + 느린 4G · A/B 인터리빙 5회 · LCP 실측 구간 비겹침 (19.2-19.5s vs 3.24-3.34s)

- 1 초기 진입 시 세션 동기화 **순서 보장** — 세션 전달이 리스너 등록 이후로 고정, 미인증 진입 경로 제거
- 2 동시 401에서 토큰 갱신 **1회로 수렴** — 웹 싱글턴 Promise + 네이티브 refreshLock의 2단 구조로 중복 갱신 경로 제거

SafeSearch

2025.12 – 2026.04

지도 위 여러 CCTV 영상을 오가며 인물의 이동 경로를 추적하는 웹

업로드 → AI 분석 → 취소·재시작으로 이어지는 긴 비동기 플로우에서 화면 상태가 실제 진행과 어긋나지 않게 상태를 설계 · 프론트엔드 1인 단독

React 19

TypeScript

Vite

TanStack Query

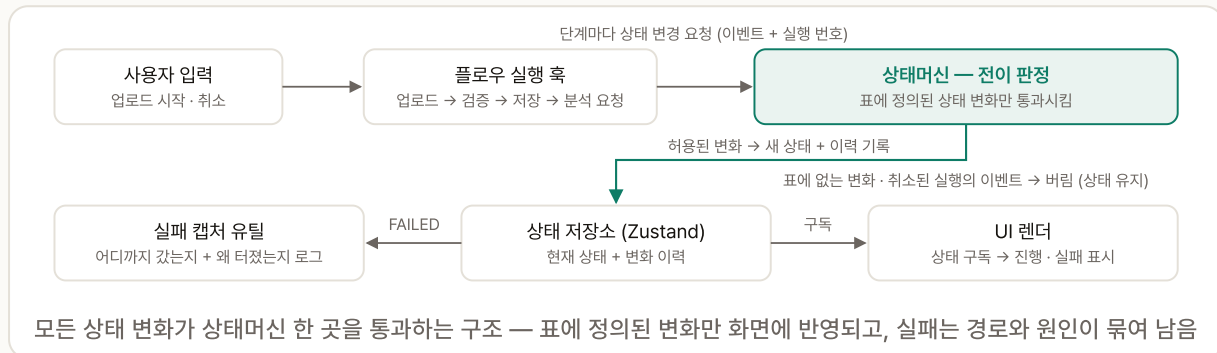
Zustand

Leaflet

Supabase

Vitest

ARCHITECTURE 전체 구조 · 데이터 흐름



PROBLEM 문제 원인

1 화면의 진행 표시를 믿을 수 없음

- "업로드 중 · 분석 대기 중" 같은 표시가 마지막에 도착한 이벤트에 라벨을 붙이는 방식이라, 지금 어느 단계인지는 검사하지 않음
- 순서가 뒤바뀌거나 단계를 건너뛴 요청도 그대로 통과해, 실제로 일어나지 않은 단계가 화면에 표시돼도 아무 신호가 없음

2 에러가 나면 로딩이 영원히 끝나지 않음

- 미리 예상하고 처리해 둔 실패만 실패 상태로 바꾸고, 예상 못 한 에러를 받아줄 장치가 없는 것이 원인
- 사용자는 끝나지 않는 로딩을 보고, 개발자는 어느 단계에서 무슨 에러였는지 알 수 없어 재현 정보가 남지 않음

3 취소하고 다시 시작하면 이전 작업의 응답이 화면을 덮어씀

- 취소한 실행과 새 실행을 구분하는 표식이 없어, 뒤늦게 도착한 이전 작업의 응답을 최신 상태로 반영한 것이 원인

SOLUTION 해결 과정

1 일어날 수 있는 상태 변화를 표로 미리 정의 — 상태머신 도입

- 화면 상태는 업로드 중 → 검증 중 → 저장 중 → 분석 대기처럼 정해진 순서로만 움직여야 하는데, 기존 코드는 응답이 도착하는 대로 상태를 바꿔 순서를 지킬 장치가 없었음
- 그래서 상태마다 "여기서 갈 수 있는 다음 단계"를 표로 먼저 적음 — 예를 들어 업로드 중 행에는 검증·취소·실패만 있어, 그 밖의 요청은 상태를 바꾸지 않고 거부(개발 환경 경고)
- 업로드 플로우 기준 일어날 수 있는 조합 144가지(상태 12 × 사건 12) 중 30가지만 허용 — 나머지 114가지는 어떤 코드가 요청해도 통과 못 함
- 타입이 모든 상태를 표의 키로 강제해, 상태를 추가하면 전이 정의를 빠뜨릴 수 없음

2 예상 못 한 에러도 전부 실패 상태로 수렴

- 플로우 전체를 감싸는 catch 경계를 뒤 어떤 예외든 FAILED 상태로 끝나게 하고, 분석 대기에 15분 상한을 뒤 조용히 멈춰 있는 경우를 없앴

3 실패하면 "어디까지 갔고 왜 터졌는지"를 한 로그로 남김

- 실패 시점까지의 상태 변화 이력과 에러 객체·화면 경로를 한 묶음으로 출력하는 캡처 유틸을 만들어, 흩어져 있던 개별 에러 로깅 대체

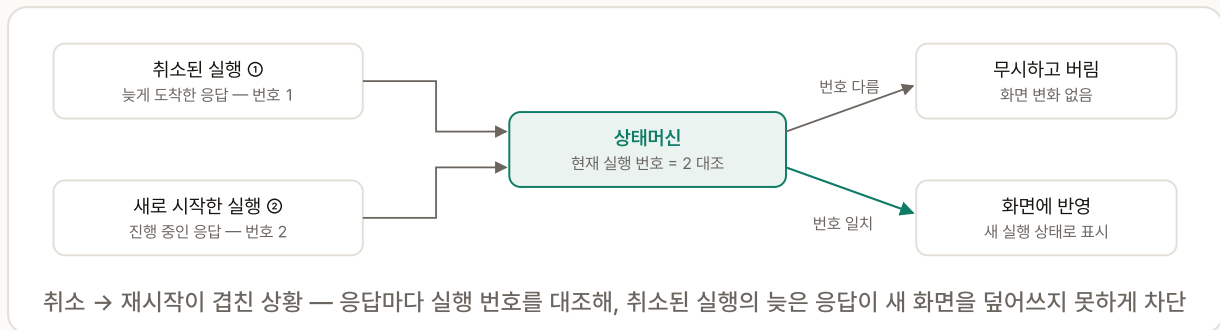
4 실행마다 번호(runId)를 매겨 이전 실행의 늦은 응답을 걸러냄

- 업로드를 취소하고 곧바로 다시 시작하면 두 실행이 겹치는데, 취소한 쪽의 서버 응답이 뒤늦게 도착하면 새 실행의 화면을 덮어서 버림
- 이를 막기 위해 플로우를 시작할 때마다 실행 번호를 매기고, 모든 상태 변경 요청에 그 번호를 실어 보냄
- 상태머신은 현재 번호와 다른 요청을 이전 실행의 것으로 판단해 버리므로, 취소·재시작을 반복해도 화면은 항상 마지막 실행만 따라감

지금 상태 \ 들어온 사건	저장소 검증	파일 행 저장	취소 요청	중단	실패
업로드 중	검증 중	거부	취소 접수	거부	실패
검증 중	거부	행 저장 중	취소 접수	중단	실패
취소 접수	거부	거부	거부	중단	실패
분석 대기 (종료)	거부	거부	거부	거부	거부
실패 (종료)	거부	거부	거부	거부	거부

전이 테이블 발췌 (영상 업로드·첨부 플로우 12×12 중 5×5) — 취소가 접수되면 진행 사건은 전부 거부되고, 종료 상태는 어떤 사건으로도 되살아나지 않음

플로우	상태	사건	전체 조합	허용	거부
영상 업로드 · 첨부	12	12	144	30	114
인물정보 등록	9	8	72	15	57



RESULT 결과

- 1 화면 표시가 실제 진행과 어긋나는 경우 제거 — 두 플로우 **12 · 9개** 상태의 흐름을 표로 명시해, 잘못된 순서의 상태 변화를 코드 레벨에서 차단
- 2 에러가 나도 무한 로딩 없이 실패 상태로 안내 — 실패 1건당 "어디까지 갔는지 + 왜 터졌는지"가 한 로그로 묶여, 재현하지 않고도 원인 특정 가능
- 3 취소·재시작을 반복해도 상태가 꼬이지 않음 — 상태 변화 로직을 React 의존 없는 순수 함수로 분리, 단위 테스트 **15개**로 검증